Code Jam to I/O for Women 2015 - Code Jam to I/O for Women

Analysis: Dreary Design

The small input is small enough to be solved by checking every possible triple of colors to see whether it meets the definition:

```
def solve1(k, v):
  total = 0
  for r in range(0, k+1):
    for g in range(0, k+1):
      for b in range(0, k+1):
        if abs(b-r) <= v and abs(b-g) <= v and abs(g-r) <= v:
          total += 1
  return total
```

The first large input is too big for this, but can still be solved using nested for loops if you incorporate some observations:

1. The tolerance values can be used to bound the range of the for statements.
2. In that case, once R and G are fixed, thereâ€™s no need to look at every possible value of B; you can instead figure out the number of valid values directly.

```
def solve2(maxvalue, tolerance):
  total = 0
  for r in range(0, maxvalue+1):
    for g in range(max(r-tolerance, 0), min(r+tolerance, maxvalue)+1):
      minb = max(r-tolerance, g-tolerance, 0)
      maxb = min(r+tolerance, g+tolerance, maxvalue)
      total += (maxb - minb + 1)
  return total
```

But this is far too slow for the second Large input, in which K can range as high as 2 billion! How can we proceed?

As is often the case in programming contests, the specified limits provide a clue. Thereâ€™s no way to iterate over 2 billion values in time, so there must be a formulaic element to the answer. Indeed, if you look at the numbers of valid values beginning with each possible value for R, a pattern emerges. For example, consider this data for K = 10, V = 3:

R	0	1	2	3	4	5	6	7	8	9	10
Valid colors	16	23	30	37	37	37	37	37	30	23	16

Once weâ€™re at least V away from either end of the range of possible values for R, the number of valid colors is always the same! This makes sense because the number of valid colors is determined by an interaction between V and the boundaries of the range. And once weâ€™re far enough away from the boundaries, that number depends only on V. Moreover, the problem is symmetric. The situation at the beginning of the range is exactly the same as at the end of the range.

These observations suggest a modification that will work for the second Large input:

```
def solve3(k, v):
  total = 0
  for r in range(0, v+1):
```

```
    subtotal = 0
    for g in range(max(r-v, 0), min(r+v, k)+1):
      minb = max(r-v, g-v, 0)
      maxb = min(r+v, g+v, k)
      subtotal += (maxb - minb + 1)
    if r == v:
      total *= 2 # take advantage of symmetry
      total += subtotal * (k - 2*v + 1)
    else:
      total += subtotal
  return total
```

One of our contestants went even farther than that! Please check out this writeup, which derives a simple, elegant formula for the answer.

Code Jam to I/O for Women 2015 - Code Jam to I/O for Women

Analysis: Googlander

Like Power Levels, this problem was inspired by pop culture: the movie Zoolander and its main character, who is unable to turn left. As he puts it: â€œIâ€™m not an ambi-turner.â€

One way to approach the problem is to construct all possible paths using, for example, a breadth-first search. Since the answer for a 10x10 grid is only 48620, this works fine for the Small data set, but itâ€™s too slow for the Large. Googlander can walk 32247603683100 paths in a 25 x 25 grid, and thatâ€™s just too many paths to keep track of! So we need a better way.

An important observation is that once Googlander leaves the line of cells heâ€™s currently walking along (without loss of generality, weâ€™ll say itâ€™s a column) by turning right, itâ€™s impossible for him to return to it (or to any cell in the grid beyond the row into which he turns). This means that once he has taken his first step out of the column, he is in a new instance of the same problem, but smaller and rotated 90 degrees to the right. This suggests a recursive solution that accounts for the fact that he can make his turn anywhere in the column:

answer(R, C) = sum from i = 1 to R of answer(C-1, i)
answer(1, anything) = answer(anything, 1) = 1

If you use memoization to avoid solving the same subproblem more than once, this recursive strategy is fast enough for the Large data set. Hereâ€™s a sample solution:

```
memo = [[-1 for i in range(26)] for j in range(26)]

def solve(r, c):
        if r == 1 or c == 1:
                return 1
        if memo[r][c] != -1:
                return memo[r][c]
        ans = 0
        for i in range(1, r+1):
                ans += solve(c-1, i)
        memo[r][c] = ans
        return ans

t = int(raw_input().strip())
for case in range(1, t+1):
        r, c = [int(a) for a in raw_input().strip().split()]
        print "Case #" + str(case) + ": " + str(solve(r, c))
```

Some of you may have noticed that the solutions to this problem are entries in Pascalâ€™s triangle (which are also the binomial coefficients). We leave it as an exercise to the reader to dig into that relationship.

Code Jam to I/O for Women 2015 - Code Jam to I/O for Women

Analysis: I/O Error

This was intended as a warm-up implementation problem to enable first-time Code Jammers to become familiar with the contestâ€™s format and environment. There are three steps to the solution:

1. Convert â€œbytesâ€ like OIOIOOOI to binary values like 01010001.
2. Convert those binary values to decimal values.
3. Convert those decimal values to characters.

Nearly all programming languages have a built-in solution for Step 3 (for example, chr() in Python.) Steps 1 and 2 required some custom code. Hereâ€™s a sample Python solution:

```
def chrfromio(b):
  s = [(1 if c == 'I' else 0) for c in b]
  v = 0
  exp = 0
  for i in range(len(s)-1, -1, -1):
    v += (2**exp) * s[i]
    exp += 1
  return chr(v)

def fromio(inp):
  ans = ''
  for i in range(len(inp)/8):
    b = inp[8*i:8*(i+1)]
    ans += chrfromio(b)
  return ans

t = int(raw_input().strip())
for case in range(1, t+1):
    d = int(raw_input().strip())
    p = raw_input().strip()
    print "Case #" + str(case) + ": " + fromio(p)
```

Some of the possible characters in this problem (single quote, double quote, backslash) were potentially troublesome because of their special interpretations, but there was no need to handle them differently since they were being produced and not interpreted.

Incidentally, the translated output for the Small input was mostly from Chapter 35 of Pride and Prejudice, one of the authorâ€™s favorite books.

Code Jam to I/O for Women 2015 - Code Jam to I/O for Women

Analysis: Power Levels

This problem was a somewhat contrived mathematical interpretation of a popular Internet meme. We know that it was a little awkward to have outputs with hundreds of exclamation points per line, but we couldn't resist. (The definition of multifactorials in the problem is real, but multifactorials donâ€™t come up too often in practice!)

Like I/O Error, this is mostly an implementation problem. There are only 9000 different multifactorials of 9000. You can calculate them all, find how many digits are in each one, and choose the one with the largest number of digits that is still smaller than the number of digits in the opponentâ€™s power level. But for the Large data set, most of those multifactorials are far too large to fit in the standard data types found in languages like C++ and Java. There were several workarounds for this:

1. Use a language extension like Javaâ€™s BigInteger that handles large numbers.

2. Use a language like Python that naturally handles arbitrarily large integers.

3. Take advantage of a useful programming contest trick: to multiply large numbers without overflowing the result, you can instead add the logarithms of those numbers and then raise the logarithm base to that power. In our case, since we only want the number of digits, you can use log base 10 and then take the ceiling of the sum. This method brings about the usual danger of small errors introduced by floating-point addition, but no multifactorial of 9000 is close enough to a power of 10 for this to pose a problem.

Hereâ€™s a sample Python solution that demonstrates the log addition method.

```
import math

def multifactorial9000(num_exc):
  ans = math.log10(9000)
  curr = 9000
  curr -= num_exc
  while curr > 0:
    ans += math.log10(curr)
    curr -= num_exc
  return math.ceil(ans)

# from number of exclamation points to number of digits
digits = [-1]*9001
for num_exc in range(1, 9001):
  digits[num_exc] = multifactorial9000(num_exc)

def solve(numdigits):
  i = 1
  while i <= 9000 and numdigits <= digits[i]:
    i += 1
  if i > 9000:
    return "..."
  return "IT'S OVER 9000" + "!"*i

t = int(raw_input().strip())
for case in range(1, t+1):
```

```
  d = int(raw_input().strip())
  print "Case #" + str(case) + ": " + str(solve(d))
```

By the way, the title is a subtle pun: the problem revolves around numbers of digits, which are closely related to powers of 10.